

Arquitectura de CoreStream

v1.0

Documento de referencia técnica de la arquitectura del sistema CoreStream.

Tabla de Contenidos

1. Resumen General
2. Diagrama de Arquitectura
3. Componentes Principales
4. Estructura de Carpetas
5. Patrones de Diseño
6. Flujos Principales
7. Integraciones Externas
8. Seguridad
9. Performance y Escalabilidad
10. Deployment

1. Resumen General

CoreStream es una plataforma de gestión de tickets/épicas orientada a equipos de desarrollo, con soporte de notificaciones en tiempo real, control de acceso basado en roles y un flujo de trabajo tipo kanban (workbench).

Stack tecnológico principal:

Capa	Tecnología
Frontend	Vue 3 + TypeScript + Vite
Backend	FastAPI (Python)
Base de Datos	PostgreSQL
Cache / Colas	Redis + ARQ

Capa	Tecnología
Deploy	Vercel (frontend) + Railway (backend)

2. Componentes Principales

2.1 Frontend — Vue 3

- **Framework:** Vue 3 (Composition API) + TypeScript + Vite como bundler.
- **Estado global:** Pinia (`stores/`) — ej. `authStore` , `ticketsStore` , `notificationsStore` .
- **Ruteo:** Vue Router (`router/`).
- **Estilos:** Tailwind CSS.
- **Comunicación HTTP:** Axios (`services/`), cliente centralizado con interceptores.
- **Comunicación en tiempo real:** WebSocket nativo gestionado por `useWebSocket.ts` .

2.2 Backend — FastAPI

- **Framework:** FastAPI (Python), servidor ASGI.
- **Routers:** separación por dominio — `auth` , `tickets` , `epics` , `subtasks` , `users` , `applications` , `notifications` , `websocket` , `analytics` .
- **Middleware:** autenticación JWT, CORS, logging.
- **Servicios:** lógica de negocio desacoplada (`notification_service.py`).
- **Modelos:** entidades SQLAlchemy.
- **Schemas:** contratos Pydantic para validación.
- **Worker:** tareas asíncronas ejecutadas por ARQ.

2.3 Base de Datos — PostgreSQL

- Motor relacional principal, acceso vía SQLAlchemy ORM.
- Migraciones versionadas con Alembic.
- Entidades principales: `users` , `roles` , `tickets` , `ticket_events` , `epics` , `subtasks` , `applications` , `documents` , `notifications` .

2.4 Redis

- **Pub/Sub:** canal `user:{id}:notifications` para eventos en tiempo real.
- **Cola de tareas:** ARQ usa Redis como backend para encolar y ejecutar jobs asíncronos.
- **Cache:** datos cacheados con TTLs configurables.

3. Patrones de Diseño Utilizados

Patrón	Dónde se aplica	Propósito
Layered Architecture	Backend (<code>routers/</code> → <code>services/</code> → <code>models/</code>)	Separación de concerns
Repository/ORM	<code>models/</code> + SQLAlchemy	Abstracción del acceso a datos
DTO / Schema Validation	<code>schemas/</code> (Pydantic)	Contratos explícitos entrada/salida
Pub/Sub	Redis + <code>redis_client.py</code> + <code>worker/tasks.py</code>	Desacoplar eventos de su consumo
Worker Queue	ARQ (<code>worker/</code>)	Procesar tareas asíncronas
Composable Pattern	Frontend <code>composables/</code>	Encapsular lógica reutilizable
Store Pattern	Pinia (<code>stores/</code>)	Estado global centralizado
RBAC	<code>models/role.py</code>	Control de autorización por rol
Event Sourcing parcial	<code>ticket_event.py</code>	Registro histórico de cambios

4. Flujos Principales

4.1 Autenticación

1. Usuario envía credenciales → POST `/api/auth/login`
2. Backend valida contra `users` (password hasheado con bcrypt)

3. Backend genera JWT con claims (user_id, role, exp)
4. Frontend almacena token en localStorage
5. authStore (Pinia) guarda estado de sesión
6. Requests posteriores envían JWT en header Authorization

4.2 Creación de Ticket

1. Usuario completa formulario en frontend
2. ticketsStore.createTicket() → POST /api/tickets
3. Router valida payload contra schema Pydantic
4. Se crea registro en `tickets` + `ticket_events`
5. `notify_*`() dispara evento
6. Worker publica en Redis → WebSocket actualiza clientes
7. Respuesta HTTP 201 retorna ticket creado

4.3 Cambio de Estado de Ticket

1. Usuario arrastra ticket o cambia estado manualmente
2. ticketsStore.updateStatus(ticketId, newStatus) → PATCH /api/tickets/{id}/status
3. Router valida transición de estado
4. Se registra `ticket_event` (auditoría)
5. Flujo de notificación en tiempo real
6. Clientes conectados se actualizan automáticamente

5. Integraciones Externas

Servicio	Uso
Vercel	Hosting y despliegue continuo del frontend
Railway	Hosting y despliegue continuo del backend + PostgreSQL/Redis
GitHub	Control de versiones, Pull Requests, CI/CD

6. Seguridad

6.1 Autenticación — JWT

- Tokens JWT generados con `python-jose[cryptography]`.
- El middleware de autenticación valida firma y expiración en cada request protegido.

6.2 Autorización — RBAC

- Roles: `ADMIN`, `TEAM_LEADER`, `DEVELOPER`.
- Cada endpoint restringe acciones según el rol del usuario autenticado.

6.3 Hashing de Contraseñas — bcrypt

- Contraseñas almacenadas con `bcrypt` (vía `passlib`), nunca en texto plano.

6.4 Otras Consideraciones

- **CORS:** configurado en middleware backend.
- **Validación de entrada:** Pydantic v2 en todos los schemas.
- **WebSocket:** autenticación del canal `/api/ws/{userId}`.
- **HTTPS/WSS:** forzado en producción vía Vercel/Railway.

7. Performance y Escalabilidad

Estado Actual

- **Backend asíncrono:** FastAPI + ARQ permiten procesar notificaciones sin bloquear.
- **WebSocket por usuario:** conexión persistente evita polling constante.
- **Frontend reactivo:** Pinia + Vue 3 minimizan renders innecesarios.

8. Deployment

8.1 Frontend — Vercel

- Build con Vite → assets estáticos servidos por Vercel.
- Despliegue continuo desde rama `main` en GitHub.

8.2 Backend — Railway

- Despliegue del backend FastAPI (contenedor Docker).
- Servicios adicionales en Railway: PostgreSQL, Redis.

8.3 Entornos

Entorno	Frontend	Backend	BD
Desarrollo local	Vite dev server	Uvicorn local	PostgreSQL Docker
Producción	Vercel	Railway	Railway managed

Documento de referencia técnica de CoreStream v1.0